SignalDesk — Project Showcase Script

Polymer Tech Expo 2026 · Part 2 of the video submission

Runs 4:15 at 150 wpm, including the demo holds. Read one bullet per breath — each bullet is one sentence. Grey lines are for filming, not for saying.

---

1 · What it is, and who it’s for 0:00 – 0:33

SHOT *Title card “SignalDesk”, then cut to the Telegram window sitting idle, before anything is typed.*

SUBTITLE *Free subscription briefing · crypto + macro in one place*

- This is **SignalDesk**.
- It’s for someone who needs to get across a market quickly, from several angles at once — **crypto and macro in one place**.
- Without spending forty minutes on headlines to find the five that matter.
- Today it’s a **Telegram bot**, and that’s what I’ll demo.
- What I’m building toward is a **free subscription site**.
- Readers subscribe, a ranked briefing is pushed every morning, and it costs them nothing.
- Which makes the list itself a distribution channel for the desk behind it.

2 · How you actually use it 0:33 – 1:16

SHOT *Record each command as its own take. Cut the loading wait every time: type → cut → reply lands. Highlight story 1’s score and its “drivers:” line. End on the Sources list at the bottom of /digest.*

SUBTITLE */top — the ranking right now · /digest — the written briefing · 08:30 daily, alerts every 15 min*

- Three commands, and that’s all of it.
- **/top** is the ranking right now — every story with its score, and the three factors that pushed it up.
- **/digest** writes the briefing: one headline, one line per story, market snapshot on top.
- And every source underneath, so you can check any claim yourself.
- But you don’t have to type anything.
- The digest goes out on its own at **half past eight every morning**.
- In between, the bot polls every fifteen minutes and pushes anything scoring **0.72 or above** immediately.
- A digest answers *what happened yesterday*; an alert answers *this can’t wait*.

3 · Where the numbers come from 1:16 – 2:23

SHOT *Two seconds on the feed list in src/fetchers/rss.py. Then /weights — record yourself tapping a subject toggle off and moving depth to Analysis, and show the two weight numbers changing. Then /why 1: HOLD FIVE SECONDS, zoomed in. This is the most important frame in the video.*

SUBTITLE *12 RSS feeds · Binance · Fear & Greed — no paid data · /why — arithmetic, not the model’s opinion*

- So where do the numbers come from?
- **Twelve RSS feeds** — CoinDesk, The Block and DL News on crypto; FT, Bloomberg and the ECB press feed on macro.
- Plus **Binance’s public ticker** and the **Fear and Greed index**.
- **None of it is paid data.**
- Every story is scored on **eight factors**: keyword tier, recency, source quality, topicality, corroboration, figures, analysis, asset.

- **/weights** hands part of that formula to the reader.
- Five subject toggles — and switching one off is a **hard filter**, not a penalty.
- Plus a depth setting that shifts weight between *numbers* and *analysis*.
- And this is the command I care most about.
- **/why 1** prints the arithmetic: every factor, its raw value, its weight, what it contributed.
- That's deliberately **not** the model explaining itself.
- Ask a model why it chose something and you get a plausible story, not the cause.
- This is the cause — and it adds up.

4 · Use of AI — and where it deliberately isn't 2:23 – 3:21

SHOT *Cut to SYSTEM_PROMPT in src/llm/digest.py and highlight "Selection is not your job." Then the terminal: python -m pytest tests/ -q, cut to the green 148 passed. Then two seconds of a real Claude Code session.*

SUBTITLE *Algorithm ranks · model only writes · "Selection is not your job" · Kimi K3 → Grok 4.3 · built with Claude Code*

- So where is the AI, and where is it deliberately not?
- **The ranking is not AI.**
- It's a deterministic algorithm — same inputs, same output — with **148 offline tests** on that path.
- The model only **writes**.
- It's handed the ranked list, and the system prompt says it plainly: *selection is not your job, don't re-order, never invent a number.*
- If every provider is down, it falls back to a template of the top headlines.
- Degraded, but still correctly ordered.
- The models are **Moonshot's Kimi K3**, falling back to **xAI's Grok 4.3**.
- Cross-vendor on purpose, because two models from one provider are a retry, not a fallback.
- And I built all of it with **Claude Code**.
- Working at the level of "here are the stages, here's why this weight is low, here's the test that has to pass."
- **My job was the logic chain.**

5 · Iterations and reflections 3:21 – 4:15

SHOT *Terminal: python scripts/show_regex_bug.py — it prints the broken and fixed rankings side by side. Let the flip land, highlight the row moving 7th → 2nd. Then CUT BACK TO YOUR FACE for the last two bullets.*

SUBTITLE *Every real bug found by reading the factor table · Next: fit the weights, ship the site*

- Two reflections.
- **What worked** was making the score explain itself.
- /why started as a nice-to-have and became my debugging tool.
- Every real bug in this codebase was found by reading the factor table next to a ranking — and **not one was caught by a test**.
- The clearest example: a regex looking for the word *exploit* couldn't match *exploited*.
- So a **sixty-two-million-dollar** protocol hack scored zero on the keyword factor and ranked seventh.
- After the fix, second.
- **What I'd improve given more time:** the weights are **reasoned, not fitted**.

- I can defend every one; none is measured.
- The next step is logging each ranking against what readers actually open, and fitting the weights to that.
- And then the site itself: subscriptions, and per-reader profiles.
- **Thank you.**

Recording checklist

Beat	What to capture	Command
2	The ranked list	/top
2	The briefing, plus the Sources list	/digest
3	The feed list, about 2 seconds	open src/fetchers/rss.py
3	Settings, with a toggle being tapped	/weights
3	Factor table — hold 5s, zoomed in	/why 1
4	System prompt: “Selection is not your job”	open src/llm/digest.py
4	The green test line	python -m pytest tests/ -q
5	The ranking flip, 7th → 2nd	python scripts/show_regex_bug.py

Before you record

- Telegram in **light** theme — dark screenshots lose the factor table after video compression.
- Zoom Telegram up two or three steps. Legibility beats fitting more in frame.
- Clear the chat first, so the demo reads top-to-bottom with nothing stale above.
- **python main.py** on live feeds if the wifi is good; **python main.py --demo** is the safe fallback, and every number in it is genuinely computed.
- 1080p minimum.

Editing

- **Cut every loading wait.** Type → cut → reply lands. That alone saves 30–40 seconds across the demo.
- Head and tail each clip tight: start on the frame the command is sent, end one beat after the reply is fully visible.
- Subtitles: key phrases only, 3–6 words, bottom third. Full-transcript subtitles make it look like a lecture.
- One hard cut back to your face for the last two bullets, so you close on a person and not a terminal.
- If you run long, drop the depth-setting bullet in Beat 3. The /why table is the argument; depth is a detail.